

SCHEDULING OF A SINGLE MACHINE TO MINIMIZE TOTAL WEIGHTED COMPLETION TIME SUBJECT TO RELEASE DATES

Lucio Bianco and Salvatore Ricciardelli

*Istituto di Analisi dei Sistemi ed Informatica del CNR
Rome, Italy*

ABSTRACT

In this paper the $n/1/r_j \geq 0/\sum_j w_j C_j$ problem under the assumptions of nonpreemptive sequencing and sequence independent processing times is investigated. After pointing out the fundamental properties, some dominance sufficient conditions among sequences are obtained and a branch and bound algorithm is proposed. Computational results are reported and discussed.

1. INTRODUCTION

The one machine scheduling problem has been studied extensively under different hypotheses and objective functions. Nevertheless, in the literature, emphasis is laid upon the case with equal ready times, no imposed due dates and flow time (or equivalent function) as objective function. In this context the problem solution is trivial since the Smith's rules [14], well-known as shortest processing time (SPT) rule and weighted shortest processing time (WSPT) rule, provide an optimal solution. According to these rules, jobs are sequenced on the basis of a preestablished order of their processing times.

In [7], [8], [12], [13], and [15], a more sophisticated cost function, such as weighted tardiness, is adopted for it turns out to be a suitable model for many real problems in which jobs may be considered available simultaneously for processing on a single machine in order to minimize the total sum of the delays with respect to their due dates. One of the most constraining assumptions is equality of the ready times of the jobs.

As a matter of fact, whenever different ready times are to be considered ($r_j \geq 0$), the previous models and the corresponding algorithms are no more adequate.

The case $r_j \geq 0$ has been recognized in research on other single machine problems, where due dates were also taken into consideration [1], [5], and [10]. However, in these works, the authors investigate the properties of a rather different cost function given by the maximum lateness (or tardiness) with weights $w_j = 1$ for all j .

In [2], [3], [4], and [11], referring to scheduling theory application to air traffic control, minimizing total weighted waiting time in one machine scheduling problem with unequal ready times has been considered.

An implicit enumeration procedure has been developed and a simplified algorithm ($w_j = 1, \forall j$) has been implemented in order to work in real time. The interested reader may address to references [11] and [4] for a complete discussion of this application. The aforementioned scheduling problem, although simple in these terms, is not trivial and it may be proved to be NP-complete [9]. The main difficulty arises from the fact that, since $r_j \geq 0$, idle times may be inserted in the optimal schedule [6].

In this paper, starting from the results obtained in previous works of ours [2], [3], [4], and [11] the $n/1/r_j \geq 0/\sum_j w_j C_j$ problem, with no preemption allowed and sequence independent processing times, is analyzed. The purpose is to establish some further dominance properties and above all to present a much improved branch and bound algorithm. Computational results, obtained with this algorithm, are also reported and discussed.

2. DEFINITIONS AND PROBLEM STATEMENT

Let $N = \{j | j = 1, 2, \dots, n\}$ be a set of jobs to be processed, one job at a time, on a single, continuously available, machine. For each job j , the ready time r_j , the processing time p_j , and the weight w_j , are given.

We call sequence on set $K \subset N$ any permutation in K which is indicated with s_k , being s_0 the sequence on the void set. Completion of all jobs requires establishing a sequence $s_n = (j_{[1]}, j_{[2]}, \dots, j_{[n]})$. When a job parameter is identified by the job's position in a given sequence rather than its index number, the position is indicated in square brackets. Thus $C_{[j]}$ means the completion time of whichever job occupies the j th position in the sequence.

Suppose now that a sequence is constructed by adding one job at a time, starting from position 1. At any step, we have a partial sequence $s_k = (j_{[1]}, j_{[2]}, \dots, j_{[k]})$ of a job set $K \subset N$. It can easily be verified that, in our setting, the completion time of such a sequence cannot be expressed only in terms of processing times regardless of the order in which jobs are scheduled, since idle times can be inserted [6]. This fact suggests some useful definitions.

DEFINITION 1: Given a partial sequence s_k , the earliest start time of a job $j \in \bar{K}$, where $\bar{K} = N - K$ is the set of jobs not sequenced, can be expressed as

$$(1) \quad t_j(s_k) = \max(r_j, C_{[k]})$$

with $t_j = r_j$, $\forall j \in N$, if $k = 0$.

DEFINITION 2: Given a partial sequence s_k , the completion time of a job $j \in \bar{K}$, $C_j(s_k)$, can be expressed as

$$(2) \quad C_j(s_k) = t_j(s_k) + p_j.$$

DEFINITION 3: Let K be any subset of N with cardinality k , then the cost function associated to s_k is

$$C^w(s_k) = \sum_{j=1}^k w_{[j]} C_{[j]}.$$

If $k = n$, $C^w(s_n)$ expresses the total weighted completion time of s_n .

Then, the problem can be formally stated as follows: Given a job set N , find a sequence s_n^* such that

$$C^w(s_n^*) \leq C^w(s_n), \quad \forall s_n \in S_N.$$

To fully understand the problem framework, it is suitable to introduce the reduced problem concept. A definition of reduced problem may be found in references [7] and [15]. However, that definition is given by considering the sequence completion time as computed regardless of the order of the jobs. As pointed out before, this is impossible in our setting; a more general definition, which also holds in our hypotheses, is the following.

DEFINITION 4: A problem is said to be reducible if a partition (N_1, N_2) of set N exists such that, if s_n^* , s_1^* , s_2^* are the optimal sequences with respect to $C^w(\cdot)$ on N , N_1 , N_2 , respectively, it is true that $s_n^* = s_1^* s_2^*$.

REMARK 1: It is easy to prove that a given problem $n/1/r_i \geq 0/\sum_j w_j C_j$ is certainly reducible if a job set N_1 and a corresponding optimal sequence s_1 exist such that completion time of s_1 is less than (or at most equal to) the earliest ready time associated with N_2 .

In [11] a simple decomposition method to transform a given problem into reduced problems is outlined.

Therefore, without loss of generality, from now on, only reduced problems will be dealt with.

DEFINITION 5: Given a partial sequence s_k , a job set $V \subset \bar{K}$ is said to be "dense" if any sequence $j_1, j_2, \dots, j_m$ on V has no inserted idle times.

It is easy to verify that

$$j \in V \text{ iff } t_j(s_k) < \min_{j \in \bar{K}} C_j(s_k).$$

DEFINITION 6: Given two sequences s_k and s_h both belonging to set S , s_k dominates s_h , if and only if, it is true that there exists an s_{n-k} such that $s_k s_{n-k} = s'_n$ belongs to S_N and

$$C^w(s'_n) \leq C^w(s_n | s_h) \text{ for all } s_n | s_h.$$

DEFINITION 7: Two sequences, s_k and s_h , are equivalent

$$\text{if } C^w(s_n^* | s_k) = C^w(s_n^* | s_h).$$

REMARK 2: On the basis of the previous definitions it follows immediately that an optimal sequence s_n^* dominates any other one, and it is equivalent to all its own partial sequences s_k , $k = 1, 2, \dots, n$.

DEFINITION 8: A sequence is said to be an ECT sequence if it satisfies the earliest completion time (ECT) rule, i.e., if $C_{[k+1]} = \min_{j \in \bar{K}} C_j(s_k)$, $0 \leq k \leq n-1$.

Ties are broken by choosing j with min t_j and further ties by choosing j with max w_j and, at last, by choosing j with min j .

DEFINITION 9: A sequence is said to be an EST sequence if it satisfies the earliest start time (EST) rule, i.e., if each actual job start time is such as

$$T_{[k+1]} = \min_{j \in \bar{K}} t_j(s_k), \quad 0 \leq k \leq n-1.$$

3. OPTIMALITY AND DOMINANCE CRITERIA

In this section a set of properties, which enhance the efficiency of the search for an optimal solution, is given. Some of these properties (specifically those resulting from Theorems 2, 3, 4, 5 and Corollary 2) have already been demonstrated in a previous paper [11]. The other properties, expressed as Theorems 1, 6, 7 and Corollaries 1 and 3, have here been carried out in order to improve the algorithm performance. Extended proofs of these further properties are reported in the Appendix.

THEOREM 1: Given a job set N and a partial sequence $s_k (k < n)$, if, for $i \in \bar{K}$

$$(a) \frac{p_i}{w_i} \leq \frac{p_j}{w_j}, \forall j \in \bar{K}$$

$$(b) t_i(s_k) \leq t_h(s_k), h \in \bar{K}$$

then $s_k i$ "dominates" $s_k h$.

Clearly, the property expressed in Theorem 1, in the case with $w_i = 1 \forall i$, can be applied in the following way: "Given a partial sequence s_k , consider, among all the remaining jobs, job i with the least processing time and then take out of consideration for $k + 1$ th position, all jobs unable to start before job i . This property, of course, appears to be efficient since earlier job i is able to start. For instance, if job i has both minimum processing time and earliest start time, no other job may be considered for possible inclusion in the $(k + 1)$ th position in the sequence.

COROLLARY 1: An EST sequence of job set N is optimal if it is ordered according to WSPT rule.

THEOREM 2: Given a job set N , a partial sequence $s_k (k < n)$ and two jobs $i, j \in \bar{K}$. If $r_j \geq C_i(s_k)$, then $s_k i$ dominates $s_k j$.

From this theorem it follows that it is possible to restrict search for optimal solution to the class of active schedules [1].

In Figure 1 an example of the application of Theorem 2 is given.

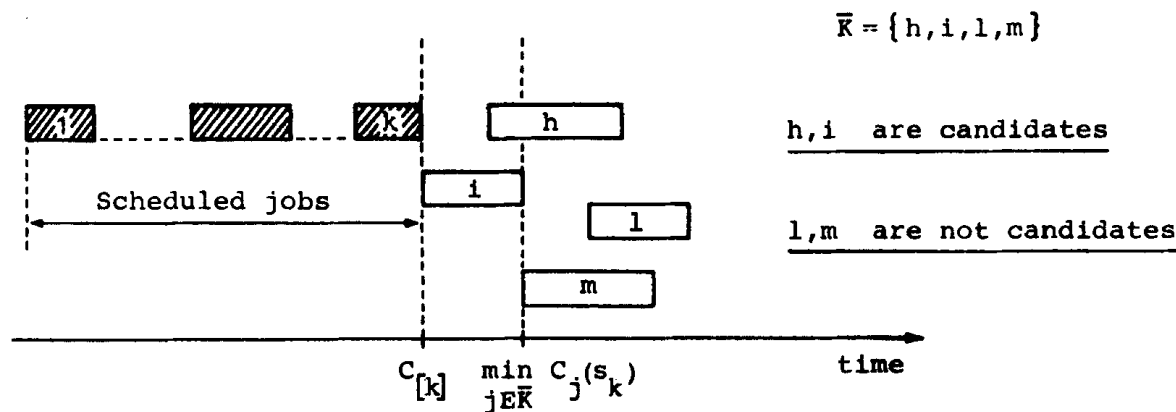


FIGURE 1. Example of application of Theorem 2: candidates to $(K + 1)$ th position in the sequence.

THEOREM 3: Given s_k and two jobs $i, j \in \bar{K}$, if

$$(a) w_i \geq w_j$$

$$(b) C_j(s_k) \geq C_i(s_k)$$

$$(c) w_i[C_j(s_k) - C_i(s_k)] + w_i p_i - w_j p_j \geq (p_j - p_i) \sum_{l \in Q} w_l$$

where Q is the set $\bar{K} - \{i, j\}$, then $s_k i$ dominates $s_k j$.

THEOREM 4: Given s_k and two jobs $i, j \in \bar{K}$, if

- (a) $w_i \geq w_j$
- (b) $C_j(s_k) \leq C_i(s_k)$
- (c) $w_i p_i - w_j p_j \geq [C_i(s_k) - C_j(s_k)] \sum_{l \in \bar{K}} w_l + \delta_{-1}(p_j - p_i) \cdot \left[\sum_{l \in \bar{K}} w_l - w_j - w_i \right]$

then $s_k i$ dominates $s_k j$.

Properties resulting from Theorems 3 and 4 may be practically utilized in order to operate a further selection within the class of active schedules. Of course, from a theoretical point of view they could also be applied within the more general class of permutation schedules. It is easily seen that, in general, the dominance concept, as expressed in Theorems 3 and 4, depends not only on the input variables (weight coefficients, processing and ready times) associated to the couple of jobs taken into consideration but also on the input variables associated to the remaining jobs (specifically on their weight coefficients or on their number if $w_i = 1 \forall i$).

THEOREM 5: Given a job set N and two partial sequences $xiyj$ and $xjyi$. If

- (a) iyj and jyi have no inserted idle times
- (b) $[C_j(x) - C_i(x)] \sum_{l \in P} w_l + [w_i - w_j] \sum_{l \in P} p_l + w_j p_i - w_i p_j \geq \delta_{-1}[t_i(x) - t_j(x)] \sum_{l \in U} w_l$

where P is the job set of iyj and U the set $N - P - \{x\}$, then $xiyj$ dominates $xjyi$.

COROLLARY 2: Given s_k and two jobs $i, j \in \bar{K}$. Let $\bar{K}$ be the set "dense," if

- (a) $w_i \geq w_j$
- (b) $t_j(s_k) \geq t_i(s_k)$
- (c) $C_j(s_k) \geq C_i(s_k)$

then $s_k i$ dominates $s_k j$.

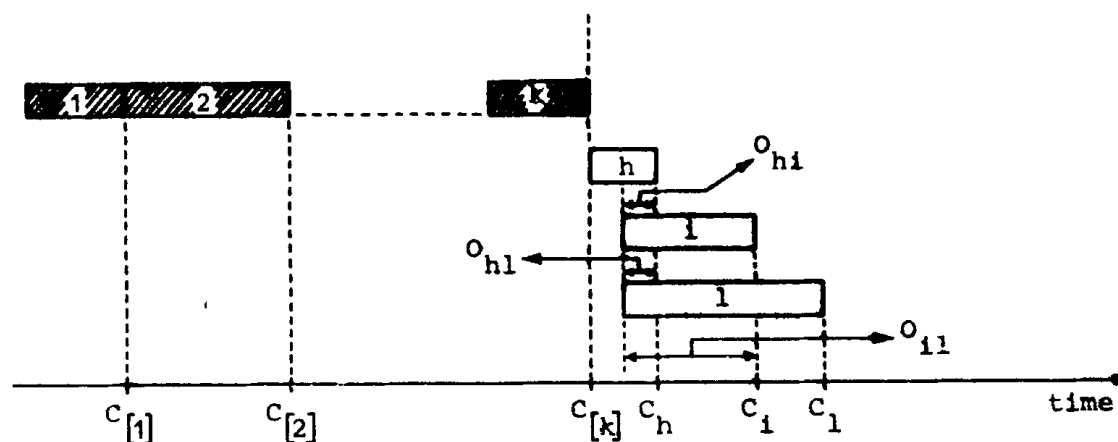
THEOREM 6: Given a job set N and a partial sequence $s_k (k < n)$, for any sequence $(s_n | s_k)$ a lower bound $\underline{C}^w(s_n | s_k) \leq C^w(s_n | s_k)$ exists, where

$$\begin{aligned} \underline{C}^w(s_n | s_k) = & C^w(s_k) + \sum_{j=1}^{n-k} w_{k+j} C_{k+j}(s_k) \\ & + \sum_{j=1}^{n-k-1} \sum_{l=i+1}^{n-k} \{ \delta_{-1}[C_{k+j}(s_k) - t_{k+l}(s_k)] \\ & - \delta_{-1}[C_{k+j}(s_k) - C_{k+l}(s_k)] \} \cdot \min(w_{k+j}, w_{k+l}), \end{aligned}$$

being $k+j$ and $k+l \in \bar{K}$ with $r_{k+j} \leq r_{k+l}$.

The first term of the lower bound provided by Theorem 6 is, simply, the value of the objective function associated with the partial sequence s_k . The second term takes into account that each job of the complementary set $\bar{K}$ cannot start before its earliest start time. The third term takes into account that processing times of the unscheduled jobs are not to be overlapped in the final sequence.

An example of application of this lower bound is reported in Figure 2.



$$\begin{aligned} \underline{C}^W(s_n/s_k) = & w[1]C[1] + \dots + w[k]C[k] + w_h c_h + w_1 c_1 + w_l c_l + o_{h1} \cdot \min(w_1, w_h) \\ & + o_{h1} \cdot \min(w_h, w_l) + o_{11} \cdot \min(w_1, w_l) \end{aligned}$$

FIGURE 2. Example of lower bound computation for a partial sequence s_k .

THEOREM 7: Let s_n^0 be a sequence on N such that, for $0 \leq k \leq n-1$

- (a) $T_{[k+1]} = C_{[k]}$
- (b) $C_{[k+1]} = \min_{j \in K} C_j(s_k)$
- (c) $w_{[k]} \geq w_{[k+1]}$

being $C_{[0]} = r_i$, $i \in N$, then s_n^0 dominates any other sequence s_n^a of the same set starting with job h such as $r_i \leq r_h$.

COROLLARY 3: An ECT sequence, defined on a job set N and ordered according to nondecreasing weights, is optimal if it has no inserted idle times and begins with the job having the smallest ready time.

4. THE PROPOSED APPROACH

To solve the problem under study, an implicit enumerative procedure is proposed, based upon a branch and bound concept. It involves a search along the branches of a tree, in which any node at level k represents a partial sequence s_k defined on a set K with cardinality k .

As a consequence, subsequent jobs in s_k correspond also to a succession of nodes encountered by going from tree root s_0 to node s_k . For each node s_k , as illustrated in Section 3, a lower bound $\underline{C}^W(s_n/s_k)$ on $C^W(s_n/s_k)$ is computed, and the current optimal solution is utilized as upper bound. At each iteration, the node being expanded is called the current node, whose last job has the lowest current value of earliest start time. A closed node is one whose correspondent partial sequence has been dominated and, therefore, it will be no longer considered. To identify dominated nodes, some elimination conditions based upon the previous theorems are then applied. In this way, the number of nodes branching from the current node can be effectively reduced.

The proposed procedure is a recursive one. In fact, whenever a partial sequence s_k on N has been fixed, the optimal sequencing problem of the jobs belonging to set $\bar{K}$ is identical to the initial problem if their ready times are shifted so that none of them is less than $C(s_k)$.

5. THE ALGORITHM

The algorithm consists of three basic phases: initialization, branching and termination. Initialization involves defining initial value of the variables, determining an initial sequence and testing it for optimality.

Branching is an iterative procedure. In each iteration, it generates the descendents of the current node, eliminates closed nodes, updates the set of job candidates to successive position, identifies, as next current node, that one having last job with lowest earliest start time, and computes the lower bound for it.

The termination phase consists of identification of the optimal sequence s^* and computation of $C^w(s^*)$.

Initialization

- (1) Let N be the job set $\{j | j = 1, 2, \dots, n\}$. Define the corresponding parameter sets:

$$R = \{r_1, r_2, \dots, r_j, \dots, r_n\}, P = \{p_1, p_2, \dots, p_n\},$$

$$W = \{w_1, w_2, \dots, w_n\} \text{ where } r_{j-1} \leq r_j \leq r_{j+1}.$$

If $r_j = r_{j+1}$ call j job with $\min p_j$

- (2) $k = 0$, $K = \emptyset$, $s_k = \emptyset$, $\bar{K} = N$, $C(s_k) = r_1$.

(3) Construct an initial solution $\tilde{s}_n$ by means of an heuristic rule (for example EST or ECT) and compute $C^w(\tilde{s}_n)$.

(4) Test initial solution for optimality by means of Corollary 1 and Corollary 3. If $\tilde{s}_n$ passes the test go to Step 22.

Branching

- (5) Determine $t_j(s_k) = \max_{j \in \bar{K}} \{r_j, C(s_k)\}$ and

$$C_m(s_k) = \min_{j \in \bar{K}} \{t_j(s_k) + p_j\}$$

- (6) Form the set

$$D_{s_k} = \{j | j \in \bar{K}, t_j(s_k) < C_m(s_k)\}$$

- (7) If $D_{s_k} = \bar{K}$ go to Step 18

- (8) Generate nodes $s_k j$, $\forall j \in D_{s_k}$

(9) For each node $s_k j$ dominated according to some of the Theorems 1, 3, 4, 5 (and Corollary 2 (if Step 8 has been reached coming from Step 19)), update set D_{s_k} by setting

$$D_{s_k} = D_{s_k} - j$$

- (10) Set $k = k + 1$
- (11) If $D_{s_{k-1}} = \emptyset$, set $k = k - 1$ and go to Step 16
- (12) Set $D_{s_{k-1}} = D_{s_{k-1}} - j'$, $s_k = s_{k-1} j'$, $K = \{s_k\}$, $\bar{K} = N - K$ with $j': t_{j'} = \min_{j \in D_{s_{k-1}}} t_j(s_{k-1})$
- (13) Compute lower bound $\underline{C}^w(s_n | s_k)$
- (14) If $\underline{C}^w(s_n | s_k) \geq C^w(\tilde{s}_n)$ go to Step 11.
- (15) Set $C(s_k) = t_{j'} + p_{j'}$ and go to Step 5.
- (16) If $k = 0$, go to Step 22.
- (17) Go to Step 11.
- (18) If $t_j(s_k) = t, \forall j \in \bar{K}$, go to Step 20.
- (19) Go to Step 8.
- (20) Form the sequence $s_k s_{n-k}$, where s_{n-k} is the sequence on $\bar{K}$ satisfying the WSPT rule.
- (21) Set $\tilde{s}_n = s_k s_{n-k}$, $C^w(\tilde{s}_n) = \underline{C}^w(s_n | s_k)$ and go to Step 11).

Termination

- (22) Set $s_n^* = \tilde{s}_n$ and $C^w(s_n^*) = C^w(\tilde{s}_n)$. STOP.

Figure 3 shows a synthetic flow-chart which illustrates the algorithm logic.

6. COMPUTATIONAL RESULTS AND CONCLUSIONS

The algorithm, previously illustrated, has been implemented on a Digital PDP 11/34 minicomputer.

Different series of tests have been performed by considering a given number of jobs $n = 10$. For each series the p_i values have been chosen uniformly distributed over a fixed interval $[1 \div 7]$ and the r_i values have been chosen uniformly distributed over five different time intervals of growing length. Moreover, for each fixed range of r_j , the weights have also been taken as uniformly distributed over ten different intervals of growing length. The results are fully reported in Table 1 while in Figures 4 and 5 are reported the diagrams showing the number of nodes generated versus $\max r_j$ and $\max w_j$, respectively. As it can be observed a significant increase in all the parameters is met whenever weight coefficients tend to be unequal. In particular, computational results show that changes in the observed parameters are relatively less dependent on the width of the weights range. Rather, these changes seem to arise whenever a mere difference among the weights is considered. Figure 4 and 5 show this behavior for the number of nodes generated.

Nevertheless, the computation time, in the worst case, does not exceed a few tens of seconds, even though a fast computer has not been used.

A last remark, involves the fact that the typical behavior (bell-shaped), observed in [4] when $w_j = 1, \forall j$, is confirmed also if w_j are distributed on a fixed interval (Figure 4).

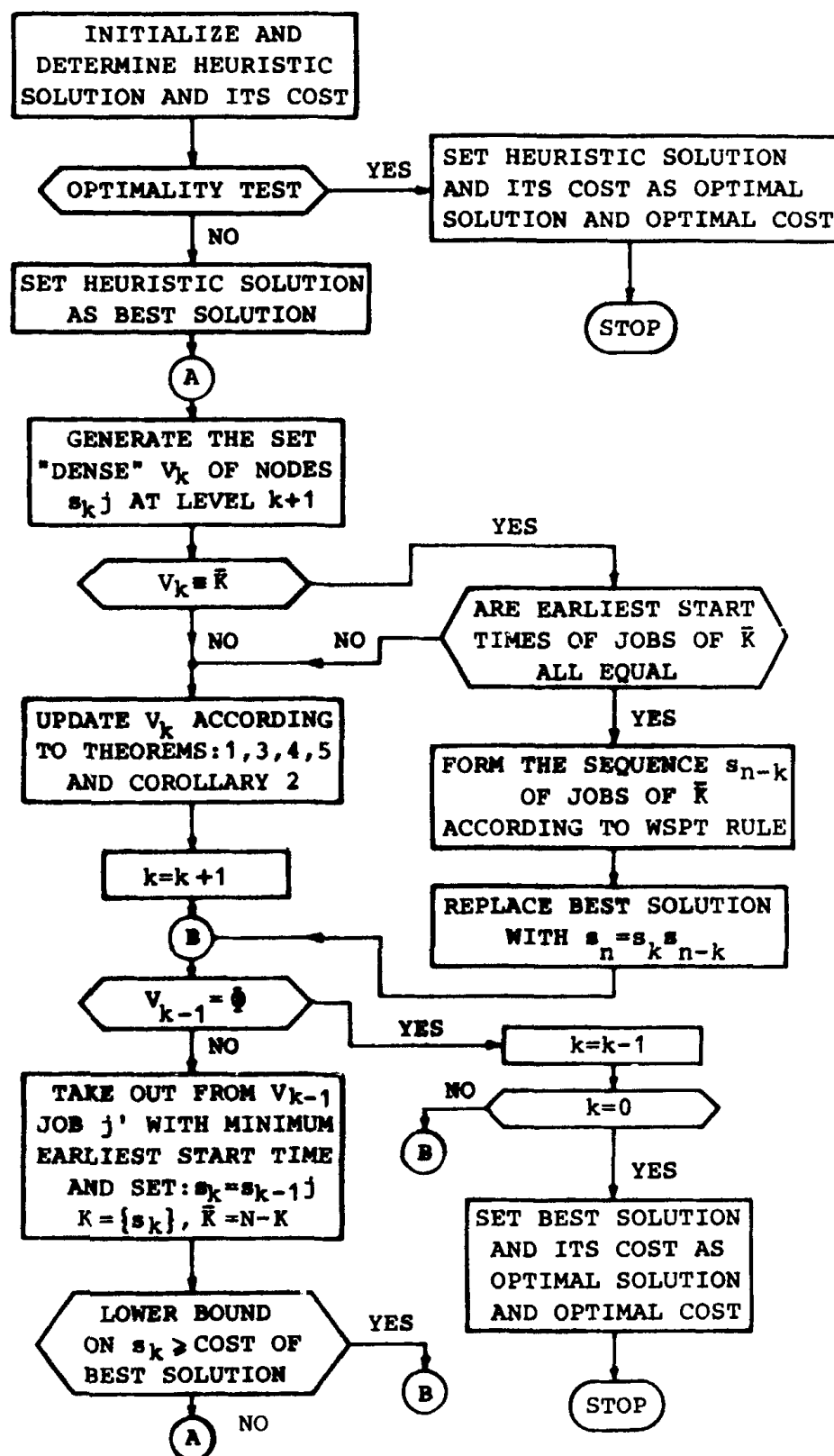


FIGURE 3. Flow chart of the algorithm.

TABLE 1 — Summary of the Results in the Case $n = 10$ and Range of $p_j = 1 \div 7$.
Number of Tests for Each Pair of Intervals Relative
to r_j and $w_j = 100$. Total Number of Tests Performed = 5000

Range of r_j Range of w_j	1 ÷ 20			1 ÷ 30			1 ÷ 40			1 ÷ 50			1 ÷ 60		
	a	b	c	a	b	c	a	b	c	a	b	c	a	b	c
1 ÷ 1	27	18	14.5	30	26	12.8	24	22	9.5	16	13	7.5	15	15	6.8
1 ÷ 1.2	44	25	22.2	42	29	17.7	35	24	12.3	28	22	9.6	24	20	8.6
1 ÷ 1.4	42	24	22.1	46	32	19.	40	28	13.3	31	22	9.8	25	20	8.6
1 ÷ 1.6	44	25	22.9	49	33	19.4	41	32	14.1	29	22	10	25	19	8.8
1 ÷ 1.8	44	25	23.8	49	30	19.4	45	32	15	29	25	9.9	24	20	8.8
1 ÷ 2.0	47	27	25	48	34	20.3	44	34	14.6	30	23	9.9	26	20	8.8
1 ÷ 2.2	47	26	25.5	55	39	22	41	34	14.9	31	25	9.8	26	21	8.9
1 ÷ 2.4	46	27	25.1	48	32	20.2	43	29	14.8	28	23	10.1	26	21	8.7
1 ÷ 2.6	48	26	25.1	52	37	21.4	40	31	13.9	27	22	9.7	25	19	8.8
1 ÷ 2.8	47	25	25.6	52	35	21.2	40	32	13.6	30	25	10.1	26	22	8.7

Note: For each range of r_j the columns a, b and c are, respectively, the average number of nodes generated, the average number of nodes examined before optimum is reached, and the computer time in seconds.

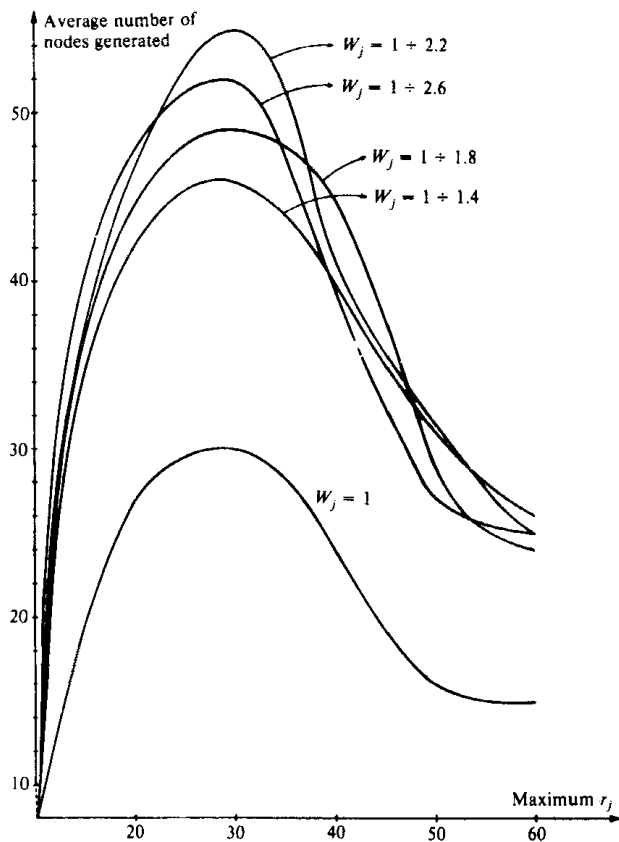


FIGURE 4. Nodes generated versus maximum ready time.

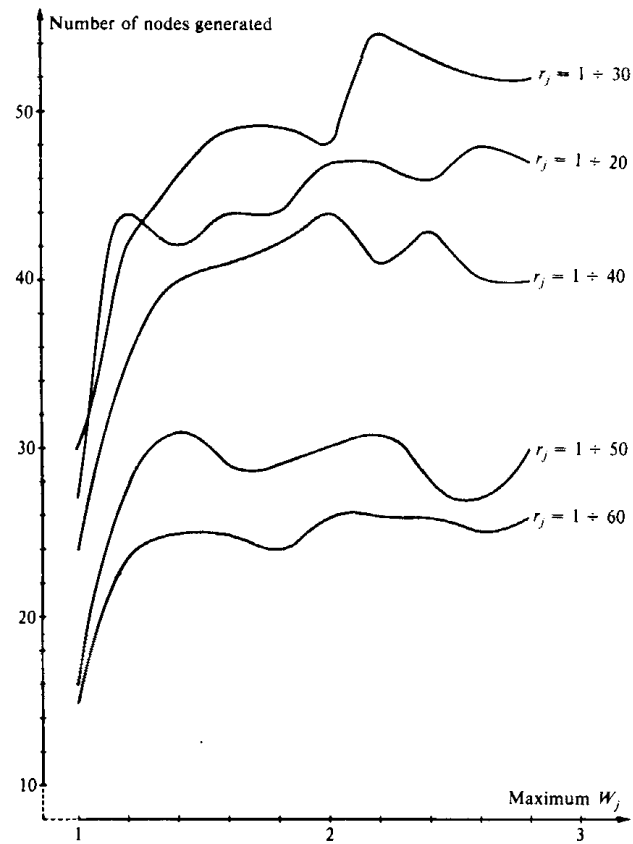


FIGURE 5. Nodes generated versus maximum weight.

In conclusion, the proposed algorithm appears to behave in a satisfactory way, at least when the number of jobs does not exceed 10.

However, to fully evaluate the algorithm performance, more extensive tests, by increasing the job number and the width of the weight ranges, by considering also processing time ranges of different width and by making use of a fast-time computer, should be provided.

REFERENCES

- [1] Baker, K.P. and Zan-Sing Su, "Sequencing with Due-Dates and Early Start Times to Minimize Maximum Tardiness," *Naval Research Logistics Quarterly*, 21, 171-176 (1974).
- [2] Bianco, L., B. Nicoletti, and S. Ricciardelli, "An Algorithm for Optimal Sequencing of Aircraft in the Near Terminal Area," *Proceedings of the 8th IFIP Conference on Optimization Techniques* (Springer Verlag, Berlin, 1977).
- [3] Bianco, L. and S. Ricciardelli, "Il Problema del Sequenziamento Ottimo Degli Aerei in Area Terminale," *Rapporto dell'Ist. di Automatica dell'Università di Roma e del C.S.S.C.C.A.-C.N.R.* - R. 78-10 (1978).
- [4] Bianco, L., S. Ricciardelli, A. Sassano, and G. Rinaldi, "The Aircraft Optimal Sequencing as an N Job One Machine Scheduling Problem," *EURO III — Amsterdam* (1979) and *Rapporto dell'Ist. di Automatica dell'Univ. di Roma e del C.S.S.C.C.A.-C.N.R.*, R. 79-36 (1979).
- [5] Bratley, P., M. Florian, and P. Robillard, "On Sequencing with Earliest Starts and Due-Dates with Application to Computing Bounds for the $(n/m/G/F \max)$ Problem," *Naval Research Logistics Quarterly*, 20, 57-67 (1973).
- [6] Conway, R.W., W.L. Maxwell, and L.W. Miller, "Theory of Scheduling" (Addison-Wesley Publishing Co., Reading, MA, 1967).
- [7] Emmons, H., "One Machine Sequencing to Minimize Certain Functions of Job Tardiness," *Operations Research*, 17, 701-715 (1969).
- [8] Fisher, M.L., "A Dual Algorithm for the One-Machine Scheduling Problem," *Mathematical Programming*, 11, 229-251 (1976).
- [9] Lenstra, J.K., A.H.G. Rinnoy Kan, and P. Brucker, "Complexity of Machine Scheduling Problems," *Studies in Integer Programming*. (North-Holland, Amsterdam, 1977).
- [10] McMahon, G., and M. Florian, "On Scheduling with Ready Times and Due Dates to Minimize Maximum Lateness," *Operations Research*, 23, 475-482 (1975).
- [11] Rinaldi, G. and A. Sassano, "On a Job Scheduling Problem with different Ready Times: Some Properties and a New Algorithm to Determine the Optimal Solution," *Rapporto dell'Ist. di Automatica dell'Università di Roma e del C.S.S.C.C.A.-C.N.R.*, 77-24 (1977).
- [12] Rinnoy Kan, A.H.G., B.J. Lageweg, and J.K. Lenstra, "Minimizing Total Costs in One-Machine Scheduling," *Operations Research*, 23, 908-927 (1975).
- [13] Shwimer, J., "On the N -Job, One-Machine, Sequence-Independent Scheduling Problem with Tardiness Penalties: a Branch-bound Solution," *Management Science*, 18, B301-B313 (1970).
- [14] Smith, W.E., "Various Optimizers for Single-Stage Production," *Naval Research Logistics Quarterly*, 3, 59-66 (1956).
- [15] Srinivasan, V., "A Hybrid Algorithm for the One-Machine Sequencing Problem to Minimize Total Tardiness," *Naval Research Logistics Quarterly*, 18, 317-327 (1971).

APPENDIX

Proof of Theorem 1:

Consider a schedule $s_n = s_k s_{n-k}$ having h in position $k + 1$ and i in position $y > k + 1$. Let s'_n be a schedule that differs from s_n only in that job i is interchanged with its immediate predecessor j in position $y - 1$. The first $(y - 2)$ jobs have exactly the same completion times under s_n and s'_n and, therefore make the same contribution to total weighted completion time.

Schedules s_n and s'_n differ only in the completion times of the $(n - y + 2)$ remaining jobs.

As far as the last $(n - y)$ jobs are concerned, which are the same in both sequences, it can be derived that

$$(1) \quad \Delta C^w(y) = \sum_{l=y+1}^n w_{[l]} C_{[l]}(s_n) - \sum_{l=y+1}^n w_{[l]} C_{[l]}(s'_n) \geq 0.$$

In fact, by assumption b), it follows that

$$\begin{aligned}C_{[y]}(s_n) &= \max(C_{[y-2]}, r_j) + p_j + p_i \\C_{[y]}(s'_n) &= \max\{(C_{[y-2]} + p_i), r_j\} + p_j,\end{aligned}$$

where j is the job in position $y - 1$. Hence, it is easy to verify that

$$C_{[y]}(s_n) \geq C_{[y]}(s'_n)$$

from which the (1) immediately follows.

Let's examine now contribution of jobs h and i . If we call $t = C_{[y-2]}(s_n)$, then their weighted completion times can be expressed by the following:

Under s_n :

$$\begin{aligned}w_j C_j &= w_{[y-1]} C_{[y-1]} = w_j \{\max(t, r_j) + p_j\} \\w_i C_i &= w_{[y]} C_{[y]} = w_i \{\max(t, r_j) + p_j + p_i\};\end{aligned}$$

Under s'_n :

$$\begin{aligned}w_j C_j &= w_{[y]} C_{[y]} = w_j \{\max(t + p_i), r_j\} + p_j \\w_i C_i &= w_{[y-1]} C_{[y-1]} = w_i (t + p_i),\end{aligned}$$

since, by assumptions b), $r_i \leq t_i \leq t_h < t$.

Two cases exist:

$$(1) \quad r_j \leq t$$

Eliminating common terms, from the (1) and assumption (a), it follows that

$$C^w(s_n) - C^w(s'_n) = \Delta C^w(y) + w_i p_j - w_j p_i \geq 0$$

$$(2) \quad r_j > t.$$

In this case

$$\begin{aligned}C^w(s_n) - C^w(s'_n) &= \Delta C^w(y) + w_j (r_j + p_j) + w_i (r_i + p_i + p_i) \\&\quad - w_j \{\max(t + p_i), r_j\} + p_j - w_i (t + p_i).\end{aligned}$$

Two subcases are to be considered:

$$(2a) \quad r_j \geq t + p_i.$$

From this and inequality (1) it follows that

$$C^w(s_n) - C^w(s'_n) = \Delta C^w(y) + w_i (r_j + p_j - t) > 0$$

$$(2b) \quad r_j < t + p_i.$$

In this subcase, it is easy to verify that

$$C^w(s_n) - C^w(s'_n) = \Delta C^w(y) + (w_j + w_i) (r_j - t) + (w_i p_j - w_j p_i) > 0$$

since the first and third terms of the sum are nonnegative, by (1) and assumption (a), respectively, and the second term is positive, being $r_j > t$.

Hence, in conclusion, s'_n represents an improvement over s_n . One could repeat the interchange operation on s'_n by shifting job i backward into position $y - 2$ and advancing the corresponding job into position $y - 1$. In this way the new sequence obtained is better than s'_n .

By induction, it is possible to repeat this operation on s'_n until job i reaches position $k + 2$. Then it follows that a partial sequence $s_k hi$ dominates every other partial sequence $s_k hj$, where $j \neq h$ and i is an element of $\bar{K}$.

To complete the proof let's consider now as schedule s_n the one with jobs h and i in the $(k + 1)$ th and $(k + 2)$ th positions, respectively.

Therefore s'_n differs from s_n in that job i is in position $k + 1$ and job h in position $k + 2$.

In this case, since $t = C_{[k]}(s_n)$ can be $< r_i$, only the completion times of s'_n must be rewritten as follows:

Under s'_n :

$$\begin{aligned} w_h C_h &= w_{[k+2]} C_{[k+2]} = w_h \{ \max[\max(t, r_i) + p_i, r_h] + p_h \} \\ w_i C_i &= w_{[k+1]} C_{[k+1]} = w_i \{ \max(t, r_i) + p_i \}. \end{aligned}$$

Three cases exist. Cases (1) and (2) have already been examined. Hence, one needs only to analyze the following:

$$(3) \quad t \leq r_i.$$

From assumption (b) and definition of t_j , it is also true that $r_i \leq r_h$. Hence,

$$\begin{aligned} C^w(s_n) - C^w(s'_n) &= \Delta C^w(k + 2) + w_h(r_h + p_h) + w_i(r_h + p_h + p_i) \\ &\quad - w_h \{ [\max(r_i + p_i), r_h] + p_h \} - w_i(r_i + p_i). \end{aligned}$$

Two subcases are then to be examined:

$$(3a) \quad r_h \geq r_i + p_i.$$

In this hypothesis it follows immediately that

$$C^w(s_n) - C^w(s'_n) = \Delta C^w(k + 2) + w_i(r_h + p_h - r_i) > 0$$

$$(3b) \quad r_h < r_i + p_i.$$

It is easy to verify that

$$C^w(s_n) - C^w(s'_n) = \Delta C^w(k + 2) + (w_h + w_i)(r_h - r_i) + (w_i p_h - w_h p_i) \geq 0$$

since all terms of the sum are nonnegative by (1), assumptions (b) and (a), respectively.

In conclusion s'_n is better than s_n .

Hence, since $s_k hi$ dominates every sequence with $s_k h$ fixed and $s_k ih$ always dominates $s_k hi$, it follows that i dominates h in position $k + 1$. Q.E.D.

Proof of Corollary 1.

It follows immediately from the previous theorem.

As far as theorems 2, 3, 4, 5 and Corollary 2 are concerned, they have been proved in [11].

Proof of Theorem 6:

It can be immediately seen that, for $k = n$, $\underline{C}^w(s_n) = C^w(s_n)$. Then it is only necessary to prove that for every k , $\underline{C}^w(s_n|s_k) \leq \underline{C}^w(s_n|s_{k+1})$.

$\underline{C}^w(s_n|s_k)$ and $\underline{C}^w(s_n|s_{k+1})$ can be expressed in the following way:

$$\underline{C}^w(s_n|s_{k+1}) = \underline{C}'(s_n|s_{k+1}) + \underline{C}''(s_n|s_{k+1})$$

$$\underline{C}^w(s_n|s_k) = \underline{C}'(s_n|s_k) + \underline{C}''(s_n|s_k)$$

where

$$\underline{C}'(s_n|s_{k+1}) = C^w(s_k) + w_{k+1}C_{k+1}(s_k) + \sum_{j=1}^{n-k-1} w_{k+1+j}C_{k+1+j}(s_{k+1});$$

$$\underline{C}''(s_n|s_{k+1}) = \sum_{j=2}^{n-k-1} \sum_{l=j+1}^{n-k} \{\delta_{-1}[C_{k+j}(s_{k+1}) - t_{k+l}(s_{k+1})]$$

$$- \delta_{-1}[C_{k+j}(s_{k+1}) - C_{k+l}(s_{k+1})]\} \min(w_{k+j}, w_{k+l});$$

$$\underline{C}'(s_n|s_k) = C^w(s_k) + w_{k+1}C_{k+1}(s_k) + \sum_{j=1}^{n-k-1} w_{k+1+j}C_{k+1+j}(s_k)$$

$$+ \sum_{l=2}^{n-k} \{\delta_{-1}[C_{k+1}(s_k) - t_{k+l}(s_k)] - \delta_{-1}[C_{k+1}(s_k) - C_{k+l}(s_k)]\}.$$

$$\cdot \min(w_{k+1}, w_{k+l});$$

$$\underline{C}''(s_n|s_k) = \sum_{j=2}^{n-k-1} \sum_{l=j+1}^{n-k} \{\delta_{-1}[C_{k+j}(s_k) - t_{k+l}(s_k)]$$

$$- \delta_{-1}[C_{k+j}(s_k) - C_{k+l}(s_k)]\} \min(w_{k+j}, w_{k+l}).$$

Therefore, we will have:

$$\underline{C}'(s_n|s_{k+1}) - \underline{C}'(s_n|s_k) = \sum_{j=1}^{n-k-1} w_{k+1+j}[C_{k+1+j}(s_{k+1}) - C_{k+1+j}(s_k)]$$

$$- \sum_{l=2}^{n-k} \{\delta_{-1}[C_{k+1}(s_k) - t_{k+l}(s_k)] - \delta_{-1}[C_{k+1}(s_k) - C_{k+l}(s_k)]\}.$$

$$\cdot \min(w_{k+1}, w_{k+l}).$$

Recalling that $l = j+1$, the generic term of the difference can be expressed as follows

$$w_{k+l}[C_{k+l}(s_{k+1}) - C_{k+l}(s_k)] - \{\delta_{-1}[C_{k+1}(s_k) - t_{k+l}(s_k)] +$$

$$- \delta_{-1}[C_{k+1}(s_k) - C_{k+l}(s_k)]\} \min(w_{k+1}, w_{k+l}).$$

Let us consider the possible cases:

$$(a) \quad C_{k+1}(s_k) < t_{k+l}(s_k)$$

$$(b) \quad t_{k+l}(s_k) \leq C_{k+1}(s_k) \leq C_{k+l}(s_k)$$

$$(c) \quad C_{k+1}(s_k) > C_{k+l}(s_k)$$

In case (a) it can be immediately verified that the generic term is equal to 0, while in case (b) and (c) it can be expressed, respectively, as follows:

$$\begin{aligned} \text{(b)} \quad & w_{k+l}[C_{k+l}(s_{k+1}) - C_{k+l}(s_k)] - [C_{k+1}(s_k) - t_{k+l}(s_k)] \min(w_{k+1}, w_{k+l}) \\ & \geq 0, \text{ since } [C_{k+l}(s_{k+1}) - C_{k+l}(s_k)] = [C_{k+1}(s_k) - t_{k+l}(s_k)]; \\ \text{(c)} \quad & w_{k+l}[C_{k+l}(s_{k+1}) - C_{k+l}(s_k)] - p_{k+l} \cdot \min(w_{k+1}, w_{k+l}) \geq 0, \\ & \text{since } [C_{k+l}(s_{k+1}) - C_{k+l}(s_k)] \geq p_{k+l}. \end{aligned}$$

Then

$$\underline{C}'(s_n | s_{k+1}) - \underline{C}'(s_n | s_k) \geq 0.$$

Let us consider now the difference $\underline{C}''(s_n | s_{k+1}) - \underline{C}''(s_n | s_k)$. Calling the generic term $c_{k+1} - c_k$, we have:

$$\begin{aligned} c_{k+1} &= \{\delta_{-1}[C_{k+j}(s_{k+1}) - t_{k+l}(s_{k+1})] - \delta_{-1}[C_{k+j}(s_{k+1}) \\ &\quad - C_{k+l}(s_{k+1})]\} \min(w_{k+j}, w_{k+l}); \\ c_k &= \{\delta_{-1}[C_{k+j}(s_k) - t_{k+l}(s_k)] - \delta_{-1}[C_{k+j}(s_k) - C_{k+l}(s_k)] \\ &\quad \cdot \min(w_{k+j}, w_{k+l}). \end{aligned}$$

The following cases can arise:

$$\begin{aligned} \text{(a)} \quad & t_{k+l}(s_k) \leq C_{k+1}(s_k) \\ \text{(b)} \quad & C_{k+1}(s_k) < t_{k+j}(s_k) \\ \text{(c)} \quad & t_{k+j}(s_k) \leq C_{k+1}(s_k) < t_{k+l}(s_k). \end{aligned}$$

Case a). According to the hypothesis

$$t_{k+j}(s_{k+1}) = t_{k+l}(s_{k+1}) = C_{k+1}(s_k)$$

and then

$$c_{k+1} = \{p_{k+j} - \delta_{-1}(p_{k+j} - p_{k+l})\} \min(w_{k+j}, w_{k+l})$$

Two subcases are possible:

$$\begin{aligned} \text{(a}_1\text{)} \quad & \text{If } p_{k+j} \leq p_{k+l}, \text{ then} \\ c_{k+1} &= p_{k+j} \{\min(w_{k+j}, w_{k+l})\}; \\ c_k &= \begin{cases} 0 \\ [C_{k+j}(s_k) - t_{k+l}(s_k)] \min(w_{k+j}, w_{k+l}). \end{cases} \end{aligned}$$

Therefore, $c_{k+1} - c_k \geq 0$.

(a₂) If $p_{k+j} > p_{k+l}$, then

$$\begin{aligned} c_{k+1} &= p_{k+l} \{\min(w_{k+j}, w_{k+l})\} \\ c_k &= \begin{cases} 0 \\ p_{k+l} \{\min(w_{k+j}, w_{k+l})\} \\ [C_{k+j}(s_k) - t_{k+l}(s_k)] \min(w_{k+j}, w_{k+l}) \end{cases} \end{aligned}$$

Therefore, $c_{k+1} - c_k \geq 0$.

Case (b). According to the hypothesis we have

$$t_{k+j}(s_k) = t_{k+j}(s_{k+1}), \quad t_{k+l}(s_k) = t_{k+l}(s_{k+1})$$

and then $c_{k+1} = c_k$.

Case (c). According to the hypothesis we have

$$t_{k+j}(s_{k+1}) = C_{k+1}(s_k) \text{ and } t_{k+l}(s_k) = t_{k+l}(s_{k+1}).$$

Three subcases exist.

$$(c_1) \quad C_{k+j}(s_{k+1}) \leq t_{k+l}(s_{k+1}) \text{ from which follows that}$$

$$c_{k+1} = c_k = 0.$$

$$(c_2) \quad t_{k+l}(s_{k+1}) < C_{k+j}(s_{k+1}) \leq C_{k+l}(s_{k+1}).$$

In this case

$$c_{k+1} = [C_{k+j}(s_{k+1}) - t_{k+l}(s_{k+1})] \min(w_{k+j}, w_{k+l});$$

$$c_k = \begin{cases} 0 \\ [C_{k+j}(s_k) - t_{k+l}(s_k)] \min(w_{k+j}, w_{k+l}) \end{cases}$$

Therefore, $c_{k+1} - c_k \geq 0$.

$$(c_3) \quad C_{k+l}(s_{k+1}) < C_{k+j}(s_{k+1}).$$

In this case

$$c_{k+1} = p_{k+l} \{\min(w_{k+j}, w_{k+l})\}$$

$$c_k = \begin{cases} 0 \\ p_{k+l} \{\min(w_{k+j}, w_{k+l}) \\ [C_{k+j}(s_k) - t_{k+l}(s_k)] \min(w_{k+j}, w_{k+l}) \end{cases}$$

Therefore, $c_{k+1} - c_k \geq 0$.

Then, also, $\underline{C}''(s_n | s_{k+1}) - \underline{C}''(s_n | s_k) \geq 0$ and hence, in general, $\underline{C}^w(s_n | s_{k+1}) \geq \underline{C}^w(s_n | s_k)$, proving that $\underline{C}^w(s_n | s_k)$ is a lower bound for $(s_n | s_k)$.

Proof of Theorem 7.

First, we show that $C_{[k]}(s_n^0) \leq C_{[k]}(s_n^a)$, $1 \leq k \leq n$. To this end let's assume that the ready times are modified, defining $r'_j = \max(r_j, r_i)$, $\forall j \in N$.

This assumption will not affect the start or completion time of any job in s_n^0 or s_n^a .

Let's construct now a new sequence s'_n , reordering, for a given k , the first k jobs in s_n^a according to their earliest start times (EST) defined on the basis of ready times r'_j . If more jobs have the same EST, priority is given to the job with the smallest completion time among them.

Let's call the partial sequence so obtained s'_k and the complete sequence s'_n ; clearly $C_{[k]}(s'_n) \leq C_{[k]}(s_n^a)$.

Since, by properties (a) and (b), s_n^0 is an ECT and EST sequence of the modified set N , it follows that, at the first position l ($1 \leq l \leq k$) in which $C_{[l]}(s_n^0) \neq C_{[l]}(s_n')$, it must be true $C_{[l]}(s_n^0) < C_{[l]}(s_n')$ and $j_{[l]}^0 \notin s_k'$.

Hence, replacing $j_{[l]}$ with $j_{[l]}^0$ will not result in an increase of $C_{[l']}(s_n')$, $l \leq l' \leq k$.

If this replacement causes jobs in s_k' following the new $j_{[l]}^0$ to lose the EST sequencing property, it is possible to reorder the set by EST without increasing $C_{[k]}(s_n')$, since $C_{[l]}(s_n')$ has been reduced. In this manner s_k' is redefined to represent the new sequence. Repeating this procedure for all l' ($l \leq l' \leq k$), comparing $C_{[l']}(s_n^0)$ and $C_{[l']}(s_n')$ and making necessary changes, as shown above, will produce a sequence identical to the partial sequence of the first k jobs in s_n^0 , without increasing $C_{[k]}(s_n^0)$. Thus, $C_{[k]}(s_n^0) \leq C_{[k]}(s_n') \leq C_{[k]}(s_n^a)$, $1 \leq k \leq n$, so proving the first statement. To complete the proof, remember that, by definition,

$$C^w(s_n^0) = \sum_{k=1}^n w_{[k]} C_{[k]}(s_n^0)$$

and

$$C^w(s_n^a) = \sum_{k=1}^n w_{[k]} C_{[k]}(s_n^a).$$

It is well known that such a sum of pairwise products of two sequences of numbers will be minimized if one sequence is arranged in increasing order and the other in decreasing order. Since the completion times $C_{[k]}$ are already in increasing order, the minimization of C^w is accomplished if sequencing is such that the weights $w_{[k]}$ are in decreasing (or at least nonincreasing) order. Therefore, being s_n^0 and s_n^a sequences defined on the same set N , from assumption (c) and previous results obtained, it follows that

$$C^w(s_n^0) \leq C^w(s_n^a)$$

which proves the theorem.

Proof of Corollary 3.

If follows immediately from the previous theorem.